

FILE INTEGRITY VERIFICATION SYSTEM

M.Sc. IT Semester 1 — Tiny Project Documentation

Technology: Python Flask + PostgreSQL

Security Technique: SHA-256 File Integrity Verification

Student Name: _____

Enrollment/Roll No.: _____

College/Institute: _____

Academic Year: 2026–27

Submitted as a Tiny Project for Internal Assessment

1. Abstract

The File Integrity Verification System is a web-based application developed using Python Flask and PostgreSQL. It allows an authenticated user to upload a file, generate its SHA-256 cryptographic hash, store file metadata, and later verify whether the file has changed. The application also maintains an activity log for important operations. The project demonstrates practical use of web development, database management and cybersecurity concepts in a small application.

2. Introduction

File integrity is important in cybersecurity, digital forensics, software distribution and document management. A file can be changed intentionally or accidentally after it is created. Cryptographic hash functions provide a compact digital fingerprint of data. In this project, SHA-256 is used to create the fingerprint. When the same file is verified later, its SHA-256 value is calculated again and compared with the original value stored in the database.

3. Problem Statement

Users need a simple way to determine whether a stored file has been modified. Manually comparing files is inefficient and may not reveal subtle changes. The proposed system provides an authenticated web interface that generates, stores and verifies SHA-256 file hashes.

4. Objectives

- Develop a simple Flask-based web application.
- Generate SHA-256 hashes for uploaded files.
- Store file metadata and hashes in PostgreSQL.
- Verify file integrity by recalculating the hash.
- Provide user authentication and owner-based access.
- Maintain an audit/activity log.
- Organize the source code as a professional GitHub repository.

5. Scope

The project is designed as a small educational application. It supports user registration, login, file upload, SHA-256 generation, verification, download, deletion and activity logging. It is not intended to replace a production digital evidence management platform. Production deployment would require additional controls such as malware scanning, hardened storage, CSRF protection, rate limiting and secure infrastructure.

6. Existing System and Limitations

A basic manual approach is to keep a copy of a file and visually compare versions. This becomes difficult for binary files and large collections. Another approach is to calculate a hash manually using command-line utilities, but that does not provide a central record or user interface. The proposed application combines hashing, storage and verification in one small web system.

7. Proposed System

The proposed system uses Flask for the web layer, PostgreSQL for metadata storage, Flask-Login for authentication and Python hashlib for SHA-256 calculation. Uploaded files are stored outside the static web directory. A server-side filename is generated to reduce filename collision and path traversal risks. The database stores the original filename, server-side filename, file size, SHA-256 value, timestamp and owner.

8. Technology Stack

Frontend: HTML, CSS and Bootstrap 5.

Backend: Python 3.11+ and Flask.

Database: PostgreSQL.

ORM: Flask-SQLAlchemy.

Authentication: Flask-Login.

Password security: Werkzeug password hashing.

File hashing: Python hashlib with SHA-256.

Version control: Git and GitHub.

9. Functional Requirements

1. Register a user.
2. Log in and log out.
3. Upload a file up to 16 MB.
4. Generate and store SHA-256.
5. View uploaded files.
6. Verify file integrity.
7. Download an owned file.
8. Delete an owned file.
9. Record important activity.

10. Non-Functional Requirements

The application should be simple to use, maintainable and responsive. Passwords should never be stored as plain text. Files should be protected from direct public access. Database credentials should be supplied through environment variables. The application should provide clear success and error messages.

11. System Architecture

Browser → Flask Application → Authentication / File Management / Hashing Logic → PostgreSQL Database.

The uploaded file is stored in the uploads directory, while metadata and the original SHA-256 digest are stored in PostgreSQL. During verification, the file is read again and a new SHA-256 digest is calculated. The two values are compared.

12. SHA-256 Working

SHA-256 is a cryptographic hash function that produces a 256-bit digest, normally represented as 64 hexadecimal characters. The application reads the file in blocks rather than loading the complete file into

memory. The digest is stored after upload. During verification, the digest is recalculated. Equal values indicate that the file contents are unchanged since the stored digest was created; different values indicate that the contents have changed or that the stored file is not the same data.

13. Database Design

users

- id: Primary key
- username: Unique username
- password_hash: Secure password hash
- created_at: Registration timestamp

file_records

- id: Primary key
- original_filename: Original uploaded name
- stored_filename: Server-side name
- file_size: File size in bytes
- sha256_hash: 64-character SHA-256 digest
- uploaded_at: Upload timestamp
- owner_id: Foreign key to users

activity_logs

- id: Primary key
- action: Activity name
- details: Activity information
- created_at: Timestamp
- user_id: Foreign key to users

14. Module Description

Authentication Module: handles registration, login and logout.

File Upload Module: receives an uploaded file and creates a safe server-side filename.

Hash Generation Module: calculates SHA-256 in 1 MB blocks.

Verification Module: calculates the current hash and compares it with the stored hash.

File Management Module: provides download and delete operations for the current user.

Activity Log Module: records upload, verification and deletion activities.

15. GitHub Repository Structure

file-integrity-verification/

■■■ app.py

■■■ requirements.txt

■■■ README.md

■■■ .env.example

■■■ .gitignore

■■■ templates/

■■■ static/css/

■■■ static/js/

■■■ models/

■■■ uploads/

■■■ screenshots/

■■■ documentation/

■■■ DOCUMENTATION.md

■■■ File_Integrity_Verification_Tiny_Project_Documentation.pdf

16. Testing

TC01 Register valid user — Expected: account created.

TC02 Login with correct credentials — Expected: dashboard opens.

TC03 Login with wrong password — Expected: error displayed.

TC04 Upload valid file — Expected: file and SHA-256 stored.

TC05 Verify unchanged file — Expected: VALID.

TC06 Verify modified file — Expected: MODIFIED.

TC07 Download own file — Expected: file downloaded.

TC08 Delete own file — Expected: file removed.

TC09 Upload file larger than 16 MB — Expected: rejected.

TC10 Access another user's record — Expected: record not accessible.

17. Advantages

- Simple and easy-to-understand cybersecurity application.
- Demonstrates Flask CRUD-style operations.
- Uses PostgreSQL for persistent metadata.
- Demonstrates practical cryptographic hashing.
- Provides authentication and audit logging.
- Suitable for a small academic project and GitHub portfolio.

18. Limitations

The application is an educational prototype. It does not include enterprise malware scanning, distributed storage, advanced role-based administration, rate limiting, cloud deployment, automated backups or a full digital forensics chain-of-custody workflow. These can be added in future versions.

19. Future Scope

Future versions can add an administrator dashboard, multiple hash algorithms, role-based access control, cloud object storage, Docker deployment, automated notifications, file version history, evidence metadata, digital signatures and a stronger audit trail.

20. Conclusion

The File Integrity Verification System demonstrates how Python Flask, PostgreSQL and cryptographic hashing can be combined to solve a practical cybersecurity problem. SHA-256 provides a digital fingerprint that can be used to detect file changes. The project is small enough for an M.Sc. IT Semester 1 Tiny Project while still demonstrating useful skills in web development, database management, authentication and cybersecurity.

21. References

1. Python Documentation — hashlib module.
2. Flask Documentation.
3. Flask-SQLAlchemy Documentation.
4. PostgreSQL Documentation.
5. Flask-Login Documentation.
6. OWASP Web Application Security guidance.

22. Detailed Test Case Table

ID	Test Case	Expected Result	Status
TC01	Valid registration	Account is created	Pass
TC02	Correct login	Dashboard is displayed	Pass
TC03	Wrong password	Login error is shown	Pass
TC04	Upload valid file	Hash and metadata stored	Pass
TC05	Verify unchanged file	VALID result	Pass
TC06	Verify changed file	MODIFIED result	Pass
TC07	Download owned file	File downloads	Pass
TC08	Delete owned file	File and record removed	Pass
TC09	Upload >16 MB	Upload rejected	Pass
TC10	Access another user's file	Access denied	Pass